Building a Live Captions Web App

A Complete Beginner's Guide to HTML, CSS, and JavaScript

This lecture walks you through every concept, every line of code, and every decision that went into building a working live-transcription web app from scratch — assuming you have never written a single line of code before. By the end you will understand not just *what* the code does, but *why* it was written that way.

Table of Contents

Chapter 1 — What Is a Web App?

Before writing a single line of code it helps to understand what we are actually building and how it ends up on your phone screen.

1.1 Websites vs. Web Apps

A **website** is mostly content: text, images, links you click to go to another page. A **web app** is interactive — it responds to what you do in real time. Gmail is a web app. Google Maps is a web app. Our live captions tool is a web app.

The distinction matters because web apps need JavaScript — the language that makes things happen dynamically, without reloading the page.

1.2 What Happens When You Open a Webpage

When you type a URL into Safari or tap a link, here is the sequence of events:

- Your browser sends a request over the internet to a server.
- The server sends back an HTML file (and sometimes CSS and JS files).
- Your browser reads the HTML and builds a visual representation called the **DOM** (Document Object Model) — a tree of every element on the page.
- The browser reads the CSS and figures out how each element should look.
- The browser runs any JavaScript, which can change the DOM in response to user actions.

Our app is special: it is a **single file**. The HTML, CSS, and JavaScript all live together in one live-captions.html file. When you open it, the browser reads everything it needs from that one file and the app works completely — no server required.

1.3 What Our App Does

Our app listens to the microphone, sends the audio to the browser's built-in speech recognition engine, and displays the recognised words on screen in real time — like subtitles, but generated live from whatever sound is around you.

Key behaviours we built:

- A Start/Stop button that toggles recording.
- A pulsing red dot that shows when the mic is active.
- Two layers of captions: **interim** (the browser's best guess so far, shown in grey) and **final** (confirmed words, shown in white).

- Automatic restart when the browser stops listening after silence.
- A buffer that trims old text so the phone doesn't slow down.
- A Clear button to wipe the transcript.
- A screen-wake lock so the phone screen stays on.
- Friendly error messages when the microphone is denied.

Chapter 2 — The Three Languages of the Web

Every webpage in existence is built from a combination of three technologies. You cannot build a web app without understanding what each one is for.

2.1 HTML — HyperText Markup Language

HTML describes **structure**. It answers the question: *what things exist on this page?*

A button, a paragraph of text, an image, a heading — each of these is described by an HTML **element**. HTML does not care what colour the button is or what happens when you click it. It only says the button exists.

2.2 CSS — Cascading Style Sheets

CSS describes **appearance**. It answers: *how does each thing look?*

CSS says the button is green, has rounded corners, is 14 pixels tall, and turns red when active. CSS has no idea what the button does — it only handles looks.

2.3 JavaScript

JavaScript describes **behaviour**. It answers: *what happens when the user does something?*

JavaScript says: when the user clicks the button, start recording from the microphone and display the words on screen. JavaScript can also change the HTML and CSS on the fly — which is how the button switches from green to red without reloading the page.

2.4 The Analogy

Think of building a house:

- **HTML** is the architect's blueprint — it defines the rooms, walls, and doors.
- **CSS** is the interior designer — it picks the paint colours, furniture, and lighting.
- **JavaScript** is the electrician and plumber — it makes the lights turn on when you flip the switch and water flow when you turn the tap.

> ■ *You can have HTML without CSS or JavaScript, but it will look like a plain text document from 1993. You cannot have CSS or JavaScript without HTML — they need something to style and control.*

Chapter 3 — HTML: Building the Skeleton

3.1 The Anatomy of an HTML File

Every HTML file begins with the same boilerplate:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title>Live Captions</title>

  <!-- CSS goes here -->

</head>

<body>

  <!-- Visible content goes here -->

  <!-- JavaScript goes here -->

</body>

</html>
```

The required wrapper every HTML file must have

<!DOCTYPE html> — Tells the browser this is a modern HTML5 document. Without this, browsers enter 'quirks mode' and render things inconsistently.

<html lang='en'> — The root element. Everything lives inside it. lang='en' tells screen readers and search engines the page is in English.

<head> — The invisible settings section. The user never sees what's in here directly, but the browser needs it.

<meta charset='UTF-8'> — Tells the browser to use the UTF-8 character encoding, which supports every language and emoji.

<meta name='viewport'> — Critical for mobile. Without this, Safari on iPhone renders the page as if it were a desktop screen and scales it down, making everything tiny. This line says: match the screen width exactly.

<body> — Everything the user sees goes here.

3.2 Tags, Elements, and Attributes

HTML is written in **tags**. A tag is a keyword wrapped in angle brackets. Most tags come in pairs — an opening tag and a closing tag:

```
<p>This is a paragraph.</p>

<button>Click me</button>

<div>This is a generic container.</div>
```

The opening tag ( ) starts the element. The closing tag ( ) ends it. The content between them is what appears on screen.

Some tags are self-closing — they have no content:

```
<meta charset="UTF-8" />

<br />    <!-- a line break -->

<hr />    <!-- a horizontal rule / divider line -->
```

Attributes are extra information added inside the opening tag:

```
<button id="toggle-btn" onclick="toggleListening()">Start</button>
```

Here, id and onclick are attributes. id gives the element a unique name JavaScript can find. onclick tells the browser which JavaScript function to call when the button is clicked.

3.3 The div and span Elements

Two elements you will see constantly:

- — A **block-level** container. It takes up the full width of the page and starts on a new line. Used to group sections of content.

- — An **inline** container. It sits inside a line of text without breaking to a new line. Used to style part of a sentence.

3.4 Our Page's HTML Structure

Our app has five distinct sections in the body. Here is each one with an explanation:

The Status Bar

```
<div id="status-bar">

  <span>

    <span id="dot"></span>

    <span id="status-text">Tap Start to begin</span>

  </span>

  <span id="lang-label">en-US</span>

</div>
```

This is the thin bar at the top of the screen. It contains the pulsing red dot (id='dot') and the status message (id='status-text'). The language label on the right (id='lang-label') shows 'en-US'.

Notice how spans are nested inside the outer div. HTML elements can contain other elements — this is called **nesting**, and it is how all page layouts are built.

The Caption Area

```
<div id="caption-area">

  <div id="transcript-box">

    <div id="final-text"></div>

    <div id="interim-text"></div>

  </div>

</div>
```

The caption-area takes up most of the screen. Inside it, transcript-box is the dark rounded rectangle that holds the text. Inside that, two divs hold the two types of captions. Both start empty — JavaScript

fills them in when speech is detected.

The Error Banner

```
<div id="error-msg" role="alert"></div>
```

This div starts hidden (CSS sets display: none). JavaScript makes it visible and fills in the text when something goes wrong, like the user denying microphone permission.

role='alert' is an accessibility attribute. Screen readers for visually impaired users will automatically announce the text when it appears, because it has been marked as an alert.

The Buttons

```
<div id="controls">

  <button id="toggle-btn" onclick="toggleListening()">Start</button>

  <button id="clear-btn" onclick="clearTranscript()">Clear</button>

</div>
```

Two buttons. The onclick attributes wire them directly to JavaScript functions. When the user taps 'Start', the browser calls toggleListening(). When they tap 'Clear', it calls clearTranscript().

The Hint

```
<div id="hint">Keep screen on in your phone settings while using</div>
```

A small text hint at the bottom of the screen, shown in dark grey so it does not distract from the captions.

Chapter 4 — CSS: Making It Look Good

4.1 Where CSS Lives

Our CSS lives inside <style> tags in the <head> section:

```
<head>

  <style>

    /* all our CSS goes here */

  </style>

</head>
```

Lines starting with /* and ending with */ are comments — the browser ignores them. They are notes for humans reading the code.

4.2 The Anatomy of a CSS Rule

```
selector {

  property: value;

  property: value;

}
```

The **selector** says *what* to style. The **properties** inside the braces say *how* to style it. Each property ends with a semicolon.

4.3 Selectors

CSS has several ways to target elements:

```
/* Element selector — targets ALL elements of that type */

button { font-size: 16px; }
```

```
/* ID selector — targets the ONE element with that id */

#toggle-btn { background: green; }


/* Class selector — targets all elements with that class */

.active { background: red; }


/* Combined — element with a class */

#dot.listening { background: red; }
```

The #dot.listening selector is important in our app. It means: 'find the element with id dot, but only apply these styles when it also has the class listening.' JavaScript adds and removes the listening class to switch the dot's appearance.

4.4 The Universal Reset

```
* {

  box-sizing: border-box;

  margin: 0;

  padding: 0;

}
```

The * selector targets *every single element* on the page. This is a standard trick to remove default browser styling. Without it, browsers add their own margins and padding to elements, which makes layouts unpredictable. We wipe everything clean and start fresh.

box-sizing: border-box changes how element sizes are calculated. By default, if you say an element is 100px wide and then add 10px of padding, the element becomes 120px wide — confusing. With border-box, padding is included in the 100px, so the element stays exactly the size you specified.

4.5 Laying Out the Page with Flexbox

We want the page to be a vertical stack: status bar at the top, captions in the middle taking all available space, buttons at the bottom. We use **Flexbox** for this.

```
body {
  background: #000;
  color: #fff;
  font-family: Arial, sans-serif;
  height: 100dvh;
  display: flex;
  flex-direction: column;
  overflow: hidden;
}
```

display: flex turns the body into a flex container. Its direct children (status bar, caption area, buttons) become flex items that can be arranged automatically.

flex-direction: column stacks children vertically (top to bottom). The default is row (left to right).

height: 100dvh — dvh means 'dynamic viewport height'. This sets the body to exactly the height of the visible screen. The d in dvh is important on mobile — it accounts for the browser's address bar appearing and disappearing, which changes the available height.

overflow: hidden prevents scroll bars from appearing if content is slightly too big.

4.6 Making the Caption Area Grow

```
#caption-area {
  flex: 1;
  display: flex;
  flex-direction: column;
  justify-content: flex-end;
  padding: 16px;
  overflow: hidden;
```

```
}
```

flex: 1 is the key line. In a flex container, if one item has flex: 1, it takes up all available space that the other items do not need. The status bar and buttons take their natural height; the caption area gets everything in between.

justify-content: flex-end pushes the caption box to the bottom of the caption area, like real subtitles — they appear near the bottom of the screen, not the top.

4.7 Colours and Fonts

```
#final-text {

  color: #fff;

  font-size: clamp(16px, 4vw, 22px);

  line-height: 1.5;

}


#interim-text {

  color: #aaa;

  font-style: italic;

  min-height: 1.5em;

}
```

color: #fff — Colours in CSS are written as hex codes. #fff is white (short for #ffffff). #aaa is a medium grey.

clamp(16px, 4vw, 22px) — This sets a responsive font size. vw means 'viewport width' — 4vw is 4% of the screen width. clamp means: use 4vw, but never go below 16px and never go above 22px. This makes the font bigger on tablets and smaller on tiny phones automatically.

min-height: 1.5em — The interim text div always takes up at least one line of height, even when empty. This prevents the layout from jumping up and down as interim text appears and disappears.

4.8 The Animated Dot

```
#dot {

  width: 10px;

  height: 10px;

  border-radius: 50%;

  background: #555;

  display: inline-block;

  transition: background 0.3s;

}


#dot.listening {

  background: #f44;

  animation: pulse 1s infinite;

}


@keyframes pulse {

  0%, 100% { opacity: 1;   }

  50%       { opacity: 0.4; }

}
```

border-radius: 50% — This is how you make a circle in CSS. Start with a square (equal width and height), then set the border radius to 50% of the width. The corners get rounded until they meet in a circle.

transition: background 0.3s — When the background colour changes, animate the change over 0.3 seconds instead of switching instantly. This gives the dot a smooth colour fade.

@keyframes pulse defines a named animation. We describe what the element looks like at different percentages through the animation (0%, 50%, 100%). The browser fills in the smooth transitions between those keyframes automatically.

animation: pulse 1s infinite applies that animation: run the 'pulse' keyframes, take 1 second per cycle, repeat forever.

4.9 The Buttons

```
#toggle-btn {

  background: #1db954;   /* Spotify green */

  color: #000;

}


#toggle-btn.active {

  background: #f44336;   /* red */

  color: #fff;

}


#clear-btn {

  background: #333;

  color: #fff;

  flex: 0 0 auto;

  width: 80px;

}
```

The Start button is green by default. When JavaScript adds the class active to it, the second rule kicks in and makes it red. When JavaScript removes active, it goes back to green. No page reload, no flicker.

flex: 0 0 auto on the Clear button means: do not grow, do not shrink, use only the width I specify (80px). This keeps the Clear button small while the Start/Stop button takes the rest of the row.

Chapter 5 — JavaScript: The Brain

5.1 Where JavaScript Lives

Our JavaScript lives inside <script> tags, placed just before the closing </body> tag:

```
<body>

  <!-- all the HTML elements -->


  <script>

    // all our JavaScript

  </script>

</body>
```

Placing the script at the bottom is important. HTML is read top to bottom. If the script were in the <head>, it would run before the HTML elements exist, and code that tries to find the button by id would fail because the button hasn't been created yet.

5.2 Variables

Variables are named containers that hold values:

```
let recognition = null;    // holds the speech recognition object

let isListening = false;   // true/false: are we currently recording?

let shouldRestart = false; // should we restart after a timeout?

let finalBuffer = '';      // the accumulated final transcript text

let restartTimer = null;   // holds a timer reference
```

let declares a variable whose value can change. const (seen elsewhere) declares a variable whose value never changes. Using const where possible makes the code easier to reason about.

The initial values matter:

- null means 'intentionally empty' — no recognition object exists yet.

- false is a boolean — it represents 'no' or 'off'.
- '' (two single quotes) is an empty string.

These five variables are the entire **state** of the app. Everything the app knows about itself at any moment is stored here.

5.3 Functions

A function is a named block of code you can run on demand:

```
function sayHello() {

  console.log('Hello!');

}


sayHello();   // calling the function runs the code inside
```

Functions can accept inputs (called **parameters**):

```
function greet(name) {

  console.log('Hello, ' + name + '!');

}


greet('World');   // prints: Hello, World!
```

And they can return outputs:

```
function add(a, b) {

  return a + b;

}


let result = add(3, 4);   // result is 7
```

5.4 Events and Event Handlers

JavaScript is **event-driven**. Code runs in response to things that happen — user clicks, keyboard presses, timers firing, speech being recognised. These 'things that happen' are called **events**.

You attach a function to an event using an **event handler**:

```
// Method 1: directly in HTML (what we use for buttons)

<button onclick="toggleListening()">Start</button>


// Method 2: in JavaScript (what we use for the wake lock)

document.getElementById('toggle-btn').addEventListener('click', () => {

  if (isListening) requestWakeLock();

});
```

The () => syntax is an **arrow function** — a shorter way to write a function that is only used in one place. These two are equivalent:

```
// Traditional function

function doSomething() { console.log('hello'); }


// Arrow function

() => { console.log('hello'); }


// Arrow function (even shorter, for single expressions)

() => console.log('hello')
```

5.5 Accessing and Changing HTML from JavaScript

JavaScript can find, read, and modify any HTML element using the DOM API:

```
// Find an element by its id

const btn = document.getElementById('toggle-btn');
```

```
// Change its text

btn.textContent = 'Stop';


// Add a CSS class

btn.classList.add('active');


// Remove a CSS class

btn.classList.remove('active');


// Check if it has a class

btn.classList.contains('active');   // returns true or false


// Change an inline style

document.getElementById('error-msg').style.display = 'block';
```

This is the bridge between JavaScript and HTML. When the user clicks Start, JavaScript finds the button by id and changes its text and class — which triggers the CSS rules for the active state.

5.6 Conditional Logic

```
if (isListening) {

  btn.textContent = 'Stop';

  btn.classList.add('active');

} else {

  btn.textContent = 'Start';

  btn.classList.remove('active');
```

```
}
```

An if statement runs code only when a condition is true. The else block runs when the condition is false. This is how the button knows which label to show.

5.7 Loops

```
for (let i = event.resultIndex; i < event.results.length; i++) {

  const transcript = event.results[i][0].transcript;

  // ...

}
```

A for loop repeats code a set number of times. Here we loop through each new speech recognition result. The loop variable i starts at resultIndex (the first new result), and increments by 1 (i++) each time, until it reaches the end of the results.

Chapter 6 — The Web Speech API

6.1 What Is an API?

An **API** (Application Programming Interface) is a set of tools that someone else built, which you can use in your own code. The browser ships with hundreds of built-in APIs — for playing sound, drawing graphics, getting the user's location, accessing the camera, and recognising speech.

Using an API means you do not have to build the hard part yourself. Speech recognition involves machine learning models trained on billions of hours of audio. We didn't build any of that. We just called the API that wraps it.

6.2 Detecting Browser Support

```
const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;


if (!SpeechRecognition) {

  document.getElementById('error-msg').style.display = 'block';

  document.getElementById('error-msg').textContent =

    'Speech recognition is not supported in this browser.';

  document.getElementById('toggle-btn').disabled = true;

}
```

window is the global browser object — everything the browser exposes to JavaScript lives on it.

The || operator means 'or'. Some older browsers implemented this API with a webkit prefix (a naming convention from when WebKit — the engine behind Safari — was adding experimental features). We check for both names and use whichever exists.

If neither exists, SpeechRecognition is undefined, which is falsy. The ! operator negates it — !undefined is true — so the if block runs, showing the error and disabling the button.

6.3 Creating and Configuring a Recognition Object

```
function buildRecognition() {
```

```
  const r = new SpeechRecognition();

  r.continuous = true;

  r.interimResults = true;

  r.lang = 'en-US';

  r.maxAlternatives = 1;

  // ... attach event handlers ...

  return r;

}
```

new SpeechRecognition() creates a new instance of the speech recogniser. Think of it like opening a new microphone session.

The four configuration options:

- continuous: true — Keep listening indefinitely instead of stopping after the first sentence.
- interimResults: true — Send partial results as the user speaks, not just after they finish a sentence.
- lang: 'en-US' — Which language/dialect to recognise. You could change this to 'es-ES' for Spanish, 'fr-FR' for French, etc.
- maxAlternatives: 1 — For each recognised phrase, only return the single best guess. You could request multiple alternatives and let the user pick the right one.

6.4 The Result Event — The Core of the App

```
r.onresult = (event) => {

  let interim = '';


  for (let i = event.resultIndex; i < event.results.length; i++) {

    const transcript = event.results[i][0].transcript;


    if (event.results[i].isFinal) {
```

```
      finalBuffer += transcript + ' ';
      if (finalBuffer.length > 1500) {
        finalBuffer = finalBuffer.slice(finalBuffer.length - 1200);
      }
    } else {
      interim += transcript;
    }
  }

  document.getElementById('final-text').textContent = finalBuffer;
  document.getElementById('interim-text').textContent = interim;
  scrollToBottom();
};
```

This function runs every time the browser has something to report. The event object it receives contains a list of results.

Understanding isFinal: The browser sends two types of results. Interim results are the browser's best guess based on what it has heard *so far*. They change constantly as more audio comes in. Final results are committed — the browser is confident and will not change them. We show interim in grey/italic and final in white.

The buffer trim: finalBuffer is a string that accumulates all final text. We check its length after every addition. If it exceeds 1500 characters, we trim it to the last 1200 characters using slice(). slice(n) returns everything from position n to the end. This prevents the string from growing to millions of characters after hours of use, which would slow down the phone.

6.5 Other Recognition Events

```
r.onstart = () => {
  setStatus('Listening...', true);
```

```
};


r.onerror = (event) => { /* handled in Chapter 8 */ };


r.onend = () => { /* handled in Chapter 7 */ };
```

onstart fires when the microphone opens and recording begins. We use it to update the status bar and dot.

onerror fires when something goes wrong.

onend fires when recognition stops — whether because the browser timed out, the user pressed Stop, or an error occurred.

Chapter 7 — The Full JavaScript Logic

7.1 The toggleListening Function

```
function toggleListening() {

  if (isListening) {

    stop(true);

  } else {

    start();

  }

}
```

This is what runs when the user taps the Start/Stop button. It checks the current state and calls the appropriate function. One button handles both actions — this pattern is called a **toggle**.

7.2 The start Function

```
function start() {

  clearError();

  recognition = buildRecognition();

  shouldRestart = true;

  isListening = true;

  updateButton();

  try {

    recognition.start();

  } catch (e) {
```

```
    showError('Could not start: ' + e.message);
    isListening = false;
    shouldRestart = false;
    updateButton();
  }
}
```

The sequence:

- clearError() — Hide any previous error message.
- buildRecognition() — Create a fresh recognition object with all event handlers attached.
- Set shouldRestart = true — This flag tells the onend handler to restart recognition if it stops due to timeout.
- Set isListening = true — Update the state.
- updateButton() — Immediately change the button to 'Stop' with red styling.
- recognition.start() inside a try/catch — Actually start the microphone. The try/catch handles any unexpected errors from the browser.

try/catch explained: Code inside try runs normally. If any line throws an error, execution jumps to catch instead of crashing. The e parameter holds the error object. This prevents the app from freezing if something unexpected happens.

7.3 The stop Function

```
function stop(userStopped) {
  shouldRestart = false;
  if (restartTimer) clearTimeout(restartTimer);
  if (recognition) {
    try { recognition.stop(); } catch (e) { }
    recognition = null;
```

```
  }

  if (userStopped) {

    isListening = false;

    setStatus('Stopped', false);

    updateButton();

  }

}
```

The userStopped parameter is a boolean. Passing true means the user pressed the button. Passing false means the app stopped recording internally (e.g. after a permission error).

The sequence:

- Set shouldRestart = false — Tell onend not to restart.
- clearTimeout(restartTimer) — If a restart was scheduled but hasn't fired yet, cancel it.
- Call recognition.stop() — Tell the browser to stop listening. Set recognition = null to free the memory.
- If the user stopped it: update isListening, the status bar, and the button.

Notice that when userStopped is false, we skip the last block. The isListening and button update will happen when onend fires — because the browser always fires onend after stop() is called.

7.4 The Auto-Restart Logic

```
r.onend = () => {

  document.getElementById('interim-text').textContent = '';

  if (shouldRestart) {

    restartTimer = setTimeout(() => {

      if (shouldRestart) {

        try { r.start(); } catch (e) { }

      }
```

```
    }, 300);
  } else {
    setStatus('Stopped', false);
    isListening = false;
    updateButton();
  }
};
```

This is the most clever piece of the app. The browser's speech recognition stops automatically after a few seconds of silence — this is a browser limitation. Without handling onend, the captions would freeze every time you stop talking.

When onend fires, we check shouldRestart:

- If true (the app is still supposed to be listening) — schedule a restart in 300 milliseconds using setTimeout.
- If false (the user pressed Stop or an error occurred) — update the UI to the stopped state.

Why 300ms? If we restart immediately (setTimeout(..., 0)), some browsers get confused by rapid start/stop cycles. A small delay gives the browser time to fully clean up before we start a new session.

Why check shouldRestart again inside the timer? The user might press Stop in the 300ms window between onend firing and the timer executing. We check again to make sure we are still supposed to restart.

setTimeout explained: setTimeout(fn, ms) schedules a function to run after a delay (in milliseconds). It returns a timer ID. clearTimeout(id) cancels that timer if it hasn't fired yet. This is exactly how we cancel the restart when the user presses Stop.

7.5 Helper Functions

```
function updateButton() {
  const btn = document.getElementById('toggle-btn');
  if (isListening) {
    btn.textContent = 'Stop';
```

```
    btn.classList.add('active');
  } else {
    btn.textContent = 'Start';
    btn.classList.remove('active');
  }
}

function setStatus(text, listening) {
  document.getElementById('status-text').textContent = text;
  document.getElementById('dot').className = listening ? 'listening' : '';
}

function clearTranscript() {
  finalBuffer = '';
  document.getElementById('final-text').textContent = '';
  document.getElementById('interim-text').textContent = '';
}

function scrollToBottom() {
  const box = document.getElementById('transcript-box');
  box.scrollTop = box.scrollHeight;
}
```

updateButton() — Reads isListening and sets the button text and CSS class accordingly.

setStatus(text, listening) — Updates the status text and the dot class in one call. className = listening ? 'listening' : '' uses a **ternary operator**: if listening is true, set className to 'listening'; otherwise set it to empty string (removing all classes).

clearTranscript() — Resets both the internal finalBuffer string AND the visible DOM elements. If you only cleared the DOM, the buffer would still have old text and the next speech event would restore it.

scrollToBottom() — Sets the scroll position of the transcript box to the maximum possible value (scrollHeight), ensuring new text is always visible.

Chapter 8 — Error Handling

Real software must handle things going wrong. The browser can send several error codes through onerror. We handle each differently:

```
r.onerror = (event) => {

  if (event.error === 'no-speech') return;


  if (event.error === 'not-allowed') {

    showError('Microphone permission denied. Allow mic access and reload.');

    stop(false);

    return;

  }


  // All other errors: fall through silently. The browser will fire

  // onend, which will restart recognition if shouldRestart is true.

};
```

no-speech

This fires when the browser is listening but hears no recognisable speech for a period. It is not a real error — silence is normal. We use return to exit the function immediately and do nothing. The browser will fire onend next, which will trigger the auto-restart.

not-allowed

This fires when the user denied microphone access (or when no microphone is available). This is a permanent error — there is no point restarting because every restart attempt will also fail. We show a human-readable message and call stop(false) to set shouldRestart = false so the auto-restart does not trigger.

Everything else

Network errors, audio capture errors, language model errors — we let these fall through silently. The browser fires onend after any error, and our auto-restart logic will handle recovery.

The showError and clearError helpers

```
function showError(msg) {

  const el = document.getElementById('error-msg');

  el.textContent = msg;

  el.style.display = 'block';

}


function clearError() {

  const el = document.getElementById('error-msg');

  el.style.display = 'none';

  el.textContent = '';

}
```

style.display = 'block' makes the error div visible (remember, CSS hid it with display: none by default). style.display = 'none' hides it again. Changing style properties directly in JavaScript overrides any CSS rule for that property.

Chapter 9 — Keeping the Screen Awake

```
let wakeLock = null;


async function requestWakeLock() {
  try {
    if ('wakeLock' in navigator) {
      wakeLock = await navigator.wakeLock.request('screen');
    }
  } catch (e) { }
}


document.getElementById('toggle-btn').addEventListener('click', () => {
  if (isListening) requestWakeLock();
});
```

9.1 Why This Is Needed

Phones dim and lock the screen after 30 seconds to 2 minutes of inactivity. But captions are passive — you are watching a movie, not touching the screen. Without a wake lock, your phone would go dark in the middle of a movie.

9.2 The Wake Lock API

navigator is a browser object containing information about the user's browser and device. 'wakeLock' in navigator checks whether this browser supports the Wake Lock API.

navigator.wakeLock.request('screen') asks the operating system to keep the screen on. It returns a promise that resolves to a lock object when granted.

9.3 async and await

Some operations take time to complete — like asking the OS for a wake lock. JavaScript handles these with **Promises**.

The async keyword marks a function as asynchronous. Inside an async function, you can use await to pause and wait for a promise to resolve before continuing.

```
// Without async/await (old style – hard to read)

navigator.wakeLock.request('screen')

  .then(lock => { wakeLock = lock; })

  .catch(e => { });


// With async/await (clean and readable)

async function requestWakeLock() {

  wakeLock = await navigator.wakeLock.request('screen');

}
```

The await line pauses requestWakeLock() until the wake lock is granted, then stores the result. The function appears synchronous (step by step) but doesn't block the rest of the page while waiting.

Wrapped in try/catch because the OS can refuse the request (e.g. low battery mode). We silently ignore refusals — a dark screen is inconvenient, not a crash.

Chapter 10 — How It All Connects

Here is the complete flow of the app from first tap to captions appearing on screen:

10.1 The Startup Flow

- Browser loads the HTML file and parses it into the DOM.
- Browser applies CSS rules to each element.
- Browser runs the JavaScript — feature-detects the Speech API, initialises all variables, attaches the click listener for the wake lock.
- The page is now idle, showing 'Tap Start to begin'.

10.2 The Start Flow

- User taps the Start button.
- onclick attribute calls toggleListening().
- isListening is false, so start() is called.
- start() calls buildRecognition(), creating a new SpeechRecognition instance with all event handlers.
- shouldRestart and isListening set to true.
- Button updates to 'Stop' (red).
- recognition.start() called — browser opens the microphone.
- Browser fires onstart — status bar shows 'Listening...', dot turns red.
- Click listener fires — requestWakeLock() called — screen stays on.

10.3 The Listening Flow (repeated many times)

- User (or TV) speaks.
- Browser's speech engine processes the audio in real time.
- Browser fires onresult with interim result — grey text appears on screen as the user speaks.
- Speaker finishes a phrase — browser fires onresult with final result.
- Text moves to finalBuffer and appears in white.
- Interim text area is cleared, ready for next phrase.
- If buffer exceeded 1500 chars, it's trimmed to 1200.
- scrollToBottom() ensures newest text is visible.

10.4 The Auto-Restart Flow

- Silence for several seconds — browser times out and stops recognition.
- Browser fires onend.
- Interim text is cleared.
- shouldRestart is true, so a 300ms timer is set.
- Timer fires — recognition.start() called again.
- Browser fires onstart — 'Listening...' again, dot still red.
- Seamless to the user — captions resume automatically.

10.5 The Stop Flow

- User taps the Stop button.
- toggleListening() called — isListening is true, so stop(true) called.
- shouldRestart = false — any pending restart is cancelled.
- recognition.stop() called — microphone closes.
- recognition = null — object cleaned up.
- Button immediately returns to 'Start' (green), status shows 'Stopped'.
- Browser fires onend — shouldRestart is false so UI stays in stopped state.

Chapter 11 — Testing the App

Writing the app is only half the job. A professional engineer also writes **tests** — automated scripts that check the app behaves correctly. We wrote 34 tests using a tool called **Playwright**.

11.1 Why Tests?

Imagine you change one line of the auto-restart code. How do you know you didn't accidentally break the Clear button? You would have to manually test every feature of the app every time you changed anything. Tests do that for you in seconds.

Tests also document behaviour. Reading a test tells you exactly what the app is supposed to do — it is living documentation.

11.2 What Is Playwright?

Playwright is a library that controls a real browser programmatically. It can open pages, click buttons, type text, and check what appears on screen — all from code, with no human operating the browser.

11.3 The Mocking Problem

Our app uses the Web Speech API, which requires a real microphone. Tests run on servers with no microphones. We solve this with a **mock** — a fake replacement for the real API that we can control from our tests:

```
// Injected into the browser before the page loads

class MockSpeechRecognition {

  constructor() {

    window.__mockRecognition = this;  // store reference for tests

  }

  start() { window.__startCount++; if (this.onstart) this.onstart(); }

  stop()  { window.__stopCount++;  }


  // Test helpers — called from test code
```

```
  _fireResult({ resultIndex, transcripts }) { ... }

  _fireError(errorCode) { if (this.onerror) this.onerror({ error: errorCode }); }

  _fireEnd()            { if (this.onend)   this.onend(); }

}


window.SpeechRecognition = MockSpeechRecognition;
```

Because this runs before the page's script, the page captures our mock instead of the real API. Then from our test code we can trigger any event: page.evaluate(() => window.__mockRecognition._fireResult(...))

11.4 What We Tested

- **Initial state** — Does the page load with 'Start' button, idle status, empty transcript?
- **Unsupported browser** — Is the error message shown and button disabled when the API is absent?
- **Starting** — Does clicking Start change the button, status text, dot, and call recognition.start()?
- **Configuration** — Is recognition set to continuous, interimResults true, lang 'en-US'?
- **Stopping** — Does clicking Stop reset everything?
- **Interim results** — Do they appear in the grey italic area?
- **Final results** — Do they move to white text? Does interim clear?
- **Accumulation** — Do multiple sentences add in order?
- **Buffer trim** — Does text over 1500 chars get trimmed to 1200?
- **no-speech error** — Is it silently ignored?
- **not-allowed error** — Is the error message shown and mic stopped?
- **Unknown errors** — Are they swallowed without crashing?
- **Auto-restart** — Does recognition restart 300ms after onend?
- **No restart after Stop** — Does onend NOT restart if user stopped?
- **Timer cancel** — Does stopping before the timer fires cancel it?
- **Clear button** — Does it wipe text and reset the buffer?

Chapter 12 — Deploying to the Internet

Deployment means making your app accessible to the world via a URL anyone can visit. A file sitting on your computer is useless to someone on a different device.

12.1 Why We Needed This

iOS's Files app preview does not allow HTML files to be opened in Safari through the share sheet. Safari is not listed as an option for local HTML files — this is a hard platform limitation Apple built in for security reasons.

The only solution is to serve the file over HTTPS from a real web server, so Safari can navigate to it via a URL.

12.2 GitHub Pages

GitHub is a website where developers store and share code. GitHub Pages is a free feature that turns any repository into a website — GitHub serves the files from their servers, and your repo gets a public URL.

The URL format is always: https://username.github.io/repository-name/filename.html

For our app: https://patrick948-stack.github.io/neetcode-submissions/live-captions.html

12.3 How to Enable It

- Go to github.com/Patrick948-stack/neetcode-submissions in Safari.
- Tap Settings (tab near the top of the repo page).
- Scroll down and tap Pages in the sidebar.
- Under Source, select 'Deploy from a branch'.
- Set Branch to 'main' and folder to '/ (root)'.
- Tap Save.
- Wait about 1 minute. Refresh the Pages settings page.
- GitHub shows a green box: 'Your site is live at https://...'
- Open that URL in Safari. The app works.

12.4 What Happens Under the Hood

When you push code to GitHub and Pages is enabled, GitHub's servers detect the new commit, copy the files to a CDN (content delivery network — servers distributed around the world), and make them

available at your github.io URL.

When Safari visits that URL, GitHub's servers send back the live-captions.html file with the correct MIME type (text/html), which tells Safari to render it as a webpage rather than display raw text.

Because the URL starts with https://, Safari grants full web API access — including the Web Speech API and microphone permission prompts.

Chapter 13 — What to Learn Next

You now understand the foundation that every web app is built on. Here is a clear path for going deeper:

13.1 Solidify the Basics

- **HTML & CSS** — freeCodeCamp.org has a free, hands-on curriculum. Do the Responsive Web Design certification.
- **JavaScript** — javascript.info is the best free reference. Read through 'The JavaScript language' section.
- **Practice** — Build small things. A to-do list. A timer. A random quote generator. Building beats reading every time.

13.2 Improve This App

- Add a language selector so users can switch between English, Spanish, French.
- Add a font size slider using an <input type='range'> element.
- Save the transcript to a file when the user taps a 'Download' button.
- Add a dark/light mode toggle.
- Show a word count.
- Highlight keywords the user specifies in the transcript.

13.3 Learn the Developer Tools

Every browser has built-in developer tools (press F12 on a desktop). The most useful panels:

- **Elements** — See the live DOM. Click any element to see and edit its HTML and CSS in real time.
- **Console** — Run JavaScript commands. See error messages. Use console.log() in your code to print values here for debugging.
- **Network** — See every file the browser downloads. Useful for understanding what a page loads.

13.4 Understand Version Control

We used **Git** to save and share our code. Git tracks every change you make to files, lets you go back to any previous version, and lets multiple people work on the same code without conflicts. **GitHub** is a website that hosts Git repositories.

Basic Git commands to learn: git init, git add, git commit, git push, git pull, git branch.

13.5 The Bigger Picture

What you built in this lesson is called a **vanilla** web app — plain HTML, CSS, and JavaScript with no frameworks. This is the foundation everything else is built on top of.

After you are comfortable here, you might explore:

- **React** — A JavaScript library for building complex UIs with reusable components. Used by Facebook, Instagram, Airbnb.
- **Node.js** — JavaScript running on a server instead of in a browser. Lets you build the backend of web apps.
- **TypeScript** — JavaScript with types added. Catches bugs before you run the code.
- **Databases** — SQL (PostgreSQL, SQLite) for structured data; MongoDB for flexible document storage.

Every professional developer started exactly where you are now. The most important thing is to keep building.

— End of Lecture —